

TAG 2 — VOM MESSEN ZUM VERÄNDERN

- **MESSEN**

Wo steht das System wirklich?

- **ISOLIEREN**

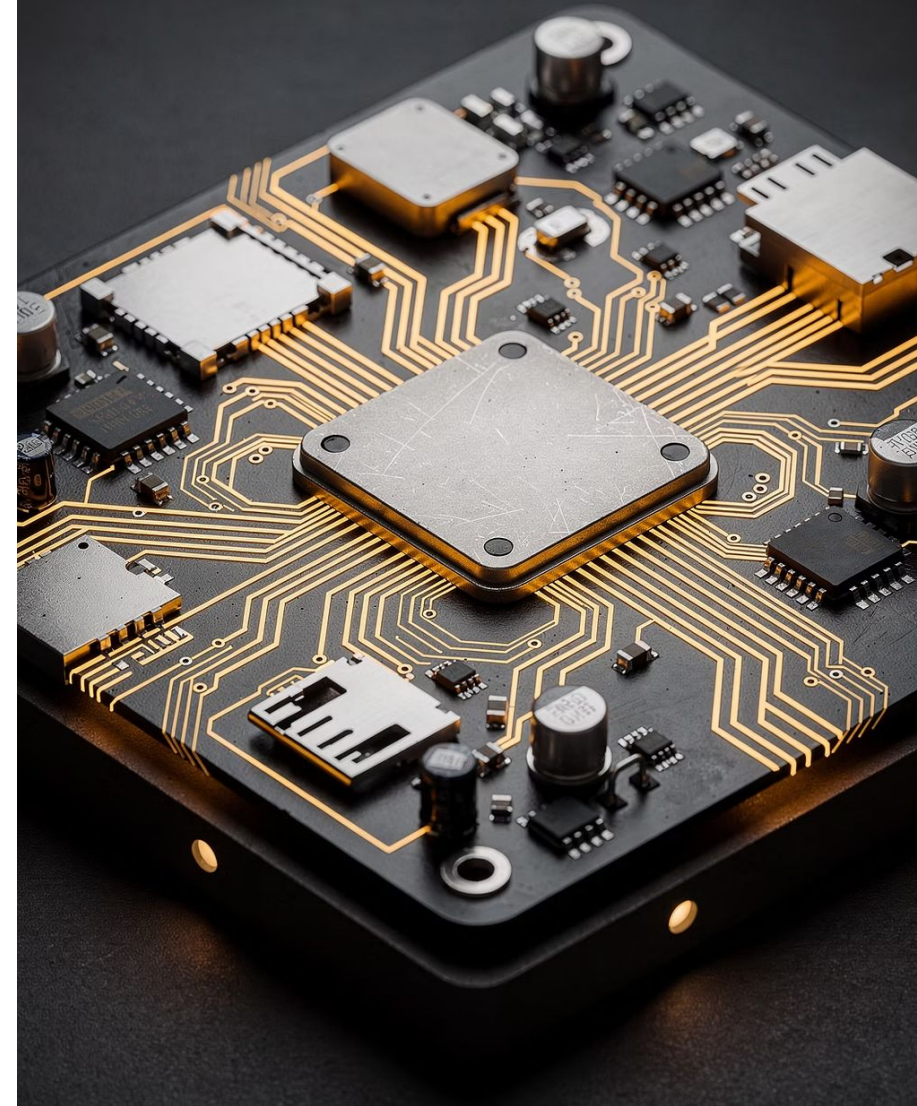
Störquellen wegnehmen

- **PRIORISIEREN**

Vorrang vergeben

- **NACHWEISEN**

Wirkung belegen, nicht behaupten



GESTERN BEREITS BENUTZT. WAS MISST ES EIGENTLICH?



WAS STELLT CYCLICTEST EIN?



WAS TUT ES DANN?



WAS LIEST ES BEIM AUFWACHEN?



WAS LANDET IN DER STATISTIK?

CYCLICTEST MISST GENAU EINE SACHE: WECKVERZUG



NUR DAS MAXIMUM IST EINE AUSSAGE

Wert	Bedeutung	Nutzen
Min	bestenfalls	interessiert niemanden
Avg	im Mittel	verschleiert das Problem
Max	schlimmstenfalls	DAS ist Ihre Deadline

Ein Mittelwert beschreibt ein System, das es nicht gibt.

CYCLICTEST SAGT DASS. RTLA SAGT WER.

CYCLICTEST

- Misst den Weckverzug
- Liefert Min, Avg, Max, Histogramm
- Zeigt, dass ein Problem existiert
- Sagt nicht, woher es kommt

RTLA TIMERLAT / RTLA OSNOISE

- Zerlegt die Latenz in ihre Anteile
- Benennt Interrupts, Threads, NMI
- Zeigt, WER die Zeit gestohlen hat
- Teil des Kernels, kein Fremdwerkzeug

Erst sichtbar machen, dann eingreifen. Nicht andersherum.

VIER DIEBE STEHLEN IHRE ZEIT



1. ENERGIESPAREN

Frequenzskalierung, C-States — die CPU muss erst aufwachen



2. INTERRUPTS

Jeder IRQ auf dem Rechenkern — fremde Arbeit auf Ihrer Zeit



3. KERNEL-HOUSEKEEPING

Timer-Tick, RCU-Callbacks, kworker — das System pflegt sich selbst



4. SPEICHER

Page Faults, Swap, Cache-Misses — der Zugriff dauert länger als gedacht

DETERMINISMUS UND ENERGIEEFFIZIENZ SIND EIN ZIELKONFLIKT

ENERGIEEFFIZIENZ

- CPU schläft, wenn möglich
- Takt fällt bei wenig Last
- Optimiert auf den Durchschnitt

DETERMINISMUS

- CPU bleibt wach und schnell
- Fester Takt, performance-Governor
- Optimiert auf den schlechtesten Fall

Wer beides verspricht, hat eins von beiden nicht gemessen.

JEDER INTERRUPT AUF IHREM KERN IST FREMDE ARBEIT

WO SIE ES SEHEN

- `/proc/interrupts` — Zeilen sind IRQs, Spalten Kerne
- Zahlen sind Zähler seit dem Systemstart
- Zweimal messen, Differenz bilden

WIE SIE ES STEUERN

- `/proc/irq/N/smp_affinity` — hexadezimale Bitmaske
- 1 = Kern 0, 2 = Kern 1, 4 = Kern 2, 8 = Kern 3
- Boot-Parameter `irqaffinity` setzt die Voreinstellung

WAS NICHT VERSCHIEBBAR IST

- Per-CPU-Interrupts, allen voran der lokale Timer

DAS SYSTEM PFLEGT SICH SELBST — AUF IHREM KERN

TIMER-TICK

- Kernel weckt sich regelmäßig selbst auf
- Bis zu 1000-mal pro Sekunde, auch im Leerlauf
- `NO_HZ_FULL` unterdrückt ihn — nur bei genau EINEM lauffähigen Thread

RCU-CALLBACKS

- Leser warten nie, dafür wird später aufgeräumt
- Das Aufräumen kommt unregelmäßig
- `rcu_nocbs` verlagert es auf andere Kerne

Zwei Threads auf einem nohz_full-Kern: der Tick kommt zurück.

DIE VIERTE EBENE — DIE, DIE SIE GESTERN NICHT GESEHEN HABEN

Von gestern bekannt:

BOOT — cmdline.txt

FIRMWARE — config.txt

LAUFZEIT — sysfs, Befehle

BUILD — DIE KERNEL-KONFIGURATION

- Wo: .config, defconfig
- Wann wirksam: erst nach dem Neubau des Kernels
- Beispiel: CONFIG_PREEMPT_RT, CONFIG_NO_HZ_FULL, CONFIG_HZ

DIE BOOT-PARAMETER, DIE ZÄHLEN

Parameter	Wirkung
<code>isolcpus=3</code>	Scheduler meidet diesen Kern
<code>nohz_full=3</code>	Kein Timer-Tick bei genau einem Thread
<code>rcu_nocbs=3</code>	RCU-Callbacks auf andere Kerne
<code>irqaffinity=0,1</code>	Voreinstellung für verschiebbare IRQs
<code>threadirqs</code>	Interrupt-Handler als Threads
<code>processor.max_cstate=1</code>	Tiefe Schlafzustände aus (x86)

⚠️ Raspberry Pi: `cmdline.txt` muss EINE einzige Zeile sein. Ein Zeilenumbruch, und alles danach wird stillschweigend ignoriert.

JETZT: LIVE-CONFIG

Wir schauen uns an, was auf diesem System tatsächlich eingestellt ist.

ISOLIERUNG IST REDUKTION, KEIN VAKUUM

WAS VERSCHWINDET

- Normale Anwendungs-Threads
- Umverteilbare Interrupts
- Timer-Tick bei nohz_full

WAS BLEIBT

- Per-CPU-Kernel-Threads
- Nicht verschiebbare Interrupts
- Alles, was Sie selbst dorthin schicken

Behaupten Sie nie, ein Kern sei leer. Weisen Sie nach, wie leer er ist.

DREI PARAMETER, DREI VERSCHIEDENE JOBS

Parameter	Was er abschaltet	Ohne ihn
<code>isolcpus=3</code>	Scheduler meidet den Kern	Normale Tasks landen dort
<code>nohz_full=3</code>	Timer-Tick auf dem Kern	Rund 1000 Unterbrechungen je Sekunde
<code>rcu_nocbs=3</code>	RCU-Callbacks auf dem Kern	Aufräumarbeit stört unregelmäßig

Nur `isolcpus` zu setzen ist der häufigste halbe Schritt.

JETZT SIE — KERN 3 ISOLIEREN

1 isolation_check.sh 3 — Zustand VOR der Änderung festhalten

2 switch-kernel.sh isolate on — dann cmdline.txt vorlesen

3 Neustart (40 bis 60 Sekunden, SSH bricht ab)

4 isolation_check.sh 3 erneut — vier Nachweise durchgehen

5 Messen unter Last auf Kern 3 und auf Kern 0 im Vergleich

VIER FRAGEN, DIE ISOLIERUNG NACHWEISEN

1

STEHT ES IN DER
BOOTZEILE?

`/proc/cmdline`

2

FÜHRT DER
KERNEL DEN KERN
ALS ISOLIERT?

`/sys/devices/system/cpu`
`/isolated`

3

LÄUFT DORT NOCH
ETWAS?

`ps -eLo pid,psr,comm`

4

KOMMEN DORT
NOCH INTERRUPTS
AN?

`/proc/interrupts` —
zweimal messen,
Differenz bilden

Drei von vier ist nicht isoliert.

DIE OBERE EBENE: SOFORT WIRKSAM, NICHT DAUERHAFT

Befehl	Wirkung
<code>taskset -c 3 ./prog</code>	Auf Kern 3 starten
<code>taskset -cp 3 PID</code>	Laufenden Prozess umziehen
<code>chrt -f 80 ./prog</code>	Mit SCHED_FIFO 80 starten
<code>ps -eLo pid,psr,class,rtprio</code>	Nachsehen, wo er wirklich läuft

Überlebt keinen Neustart. Genau deshalb ist es Ihre Experimentierfläche.

DER STANDARD-SCHEDULER IST FAIR. DAS IST UNSER PROBLEM.

CFS — COMPLETELY FAIR SCHEDULER

- Ziel: jeder kommt irgendwann dran
- Optimiert Durchsatz und Reaktionsgefühl
- Niemand verhungert
- Niemand hat garantiert Vorrang

SCHED_FIFO — ECHTZEIT-SCHEDULING

- Ziel: der Wichtigste kommt sofort dran
- Optimiert den schlechtesten Fall
- Läuft bis er blockiert oder verdrängt wird
- Wer unten steht, kann verhungern

Isolierung schafft einen leeren Raum. Scheduling entscheidet, wer ihn betritt.

FÜNF POLICIES, DREI DAVON BRAUCHEN SIE

Policy	Verhalten	Wann
SCHED_OTHER	CFS, fair, Nice-Wert	alles Normale
SCHED_FIFO	läuft bis blockiert oder verdrängt	Ihr Standardfall
SCHED_RR	wie FIFO plus Zeitscheibe unter Gleichen	selten nötig
SCHED_DEADLINE	Frist und Rechenbedarf statt Wichtigkeit	wenn Deadline bekannt
SCHED_IDLE	nur wenn sonst nichts will	Hintergrundarbeit

⊗ FIFO hat keine Zeitscheibe. Eine Endlosschleife hält den Kern für immer.

ERST DAS EINFACHE MODELL: WIE EIN RTOS TASKS ABARBEITET

RTOS (Z. B. FREERTOS)

- Jede Task hat eine feste Prioritätszahl
- Scheduler nimmt immer die höchste lauffähige Task
- Task läuft bis sie blockiert oder verdrängt wird
- Kein Speicherschutz, kein Kernel-Userspace-Schnitt

LINUX MIT SCHED_FIFO

- Dasselbe Prinzip — für Threads mit Echtzeit-Policy
- Daneben laufen weiterhin alle normalen Prozesse
- Der Kernel selbst kann dazwischenkommen
- Dafür: Netzwerk, Dateisystem, Bibliotheken

Linux ist nicht anders. Linux ist dasselbe Prinzip plus alles andere.

PRIORITÄT IST EINE AUSSAGE ÜBER DEADLINES, KEIN WUNSCH

Bereich	Wofür	Hinweis
99	Kernel-Threads, Watchdog	Finger weg
80 – 90	Harte Deadline, kurze Laufzeit	Ihr Regelkreis
50 – 79	Echtzeit mit Toleranz	Messwerterfassung
10 – 49	Echtzeitnah, unkritisch	Logging, Weiterleitung
SCHED_OTHER	Alles Übrige	Netzwerk, UI, Auswertung

Wer alles auf 99 setzt, hat die Reihenfolge nur anders zufällig gemacht.

REGELKREIS ALLE 10 MS ODER SICHERHEITSÜBERWACHUNG ALLE 100 MS — WER BEKOMMT DIE HÖHERE PRIORITÄT?

RATE MONOTONIC

Die kürzere Periode bekommt die
höhere Priorität

WICHTIGKEIT ≠ DRINGLICHKEIT

Häufiger schlägt wichtiger

DIE ÜBERWACHUNG HAT 100 MS ZEIT

Ihre Frist ist die Größe, die zählt

JETZT: TERMINAL

Wer läuft auf diesem System mit welcher Priorität? Suchen Sie mir die 99er.

**KANN EIN THREAD MIT PRIORITÄT
80 JEMALS AUF EINEN THREAD MIT
PRIORITÄT 10 WARTEN?**

Denken Sie nach.

EIN ECHTZEIT-THREAD WIRD ANDERS ERZEUGT ALS EIN NORMALER

01

ATTRIBUTE SETZEN

Policy SCHED_FIFO, Priorität, PTHREAD_EXPLICIT_SCHED

02

THREAD ERZEUGEN

Mit diesen Attributen — nicht nachträglich

03

AFFINITÄT SETZEN

Auf welchem Kern soll er laufen

04

IM THREAD

Speicher festnageln, dann zyklisch arbeiten

DER HÄUFIGSTE FEHLER

- Thread erbt die Priorität des Erzeugers
- PTHREAD_EXPLICIT_SCHED nicht gesetzt
- Ergebnis: Echtzeit-Thread läuft als normaler Thread

Für den Takt: monotone Uhr. Für den Zeitstempel: Wanduhr.

MUTEX UND SEMAPHORE LÖSEN ZWEI VERSCHIEDENE PROBLEME

MUTEX — BESITZ

- Schützt eine gemeinsame Ressource
- Genau ein Besitzer, nur der gibt frei
- Kennt seinen Halter — Vererbung möglich
- Frage: Wer darf gerade zugreifen?

SEMAPHORE — ZÄHLEN UND SIGNALISIEREN

- Zählt freie Plätze oder gibt ein Signal
- Kein Besitzer, jeder darf hochzählen
- Kennt keinen Halter — keine Vererbung
- Frage: Ist etwas passiert? Ist noch Platz?

Ein Semaphore mit Zählwert 1 ist KEIN Mutex.

MARS, JULI 1997

4. JULI

Pathfinder landet, alles läuft

TAG 3

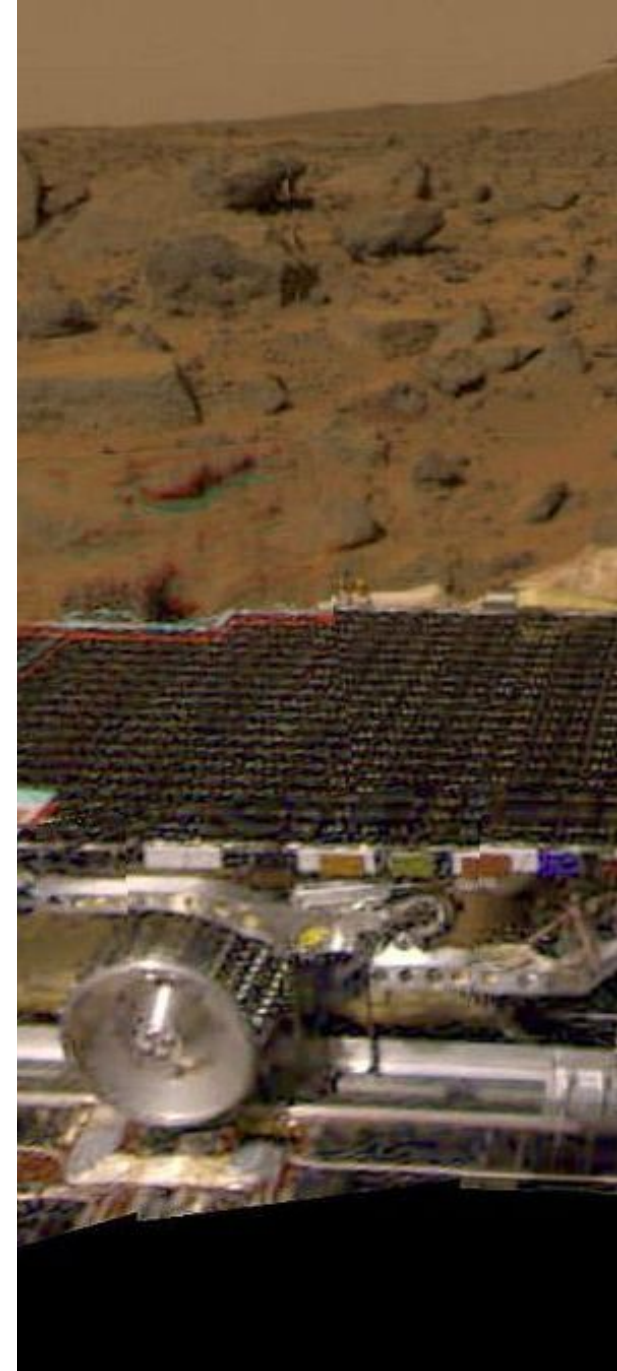
Das System startet sich selbst neu

DANACH

Neustarts wiederholen sich, Messdaten gehen verloren

DIAGNOSE

Nicht der Weltraum. Ein Mutex.



ZWEI FORMEN — NUR EINE IST EIN FEHLER

BESCHRÄNKT — UNVERMEIDBAR, PLANBAR

- Hoch wartet auf Niedrig
- Wartezeit = Länge des kritischen Abschnitts
- Abschätzbar, einplanbar
- Kein Fehler

UNBESCHRÄNKT — DER FEHLER

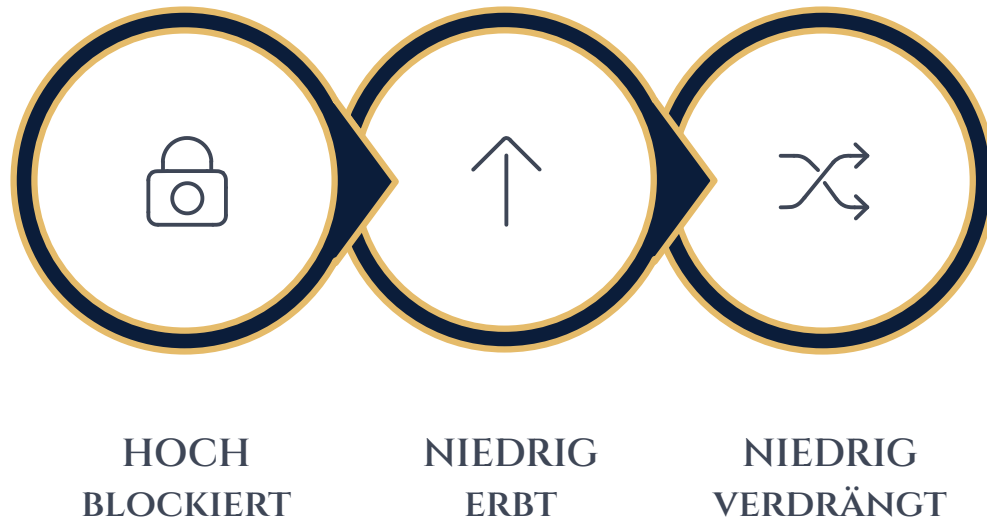
- Ein dritter Thread verdrängt den Halter
- Wartezeit hängt von einem Unbeteiligten ab
- Nicht nach oben abschätzbar
- Ihr Worst Case existiert nicht mehr

Unbeschränkt heißt nicht lange. Es heißt: nicht abschätzbar.

JETZT: WIR BAUEN ES NACH

Drei Threads. Ein Mutex. Ein Kern. Auf Ihrem Pi.

PRIORITY INHERITANCE: DER HALTER ERBT DIE DRINGLICHKEIT



WAS ES KOSTET

- Laufzeit bei jedem Sperrvorgang
- Nicht pauschal überall einschalten
- Beste Lösung bleibt: kein geteiltes Mutex

Auf dem Pathfinder war diese Option vorhanden — und abgeschaltet.

DER SPEICHER, DER NICHT DA IST, WENN SIE IHN BRAUCHEN

DAS PROBLEM

- `malloc` gibt ein Versprechen, keinen Speicher
- Der erste Zugriff löst einen Page Fault aus
- Der Kernel holt nach — unvorhersehbar lange

DIE MASSNAHME

- `mlockall` mit `MCL_CURRENT` und `MCL_FUTURE`
- Speicher wird festgenagelt, kein Auslagern

DIE REGEL

- In der Echtzeitschleife wird nicht mehr alloziert
- Speicher in der Startphase holen UND anfassen

JETZT: LERNPROJEKT STUFE 2

Ihr Programm von gestern. Heute deterministisch.

LERNPROJEKT STUFE 2 — EINE ÄNDERUNG, EINE MESSUNG

1 M0

Basismessung — Ihre Einstellungen von gestern

2 M1

Nur die Scheduling-Policy ändern, messen

3 M2

Zusätzlich den Kern festlegen, messen

4 M3

Zusätzlich den Speicher festnageln, messen

RAHMENBEDINGUNGEN

- Jede Messung mindestens 60 Sekunden
- Jede Messung unter Last
- Maxima vergleichen, nicht Mittelwerte

⚠ Eine der drei Änderungen bringt kaum etwas. Welche — und warum?

STUFE 2: EINE ÄNDERUNG, EINE MESSUNG

01

BASISMESSUNG

Mit den Einstellungen von gestern

02

NUR DIE POLICY ÄNDERN

Messen

03

NUR DEN KERN FESTLEGEN

Messen

04

NUR DEN SPEICHER FESTNAGELN

Messen

ABNAHME

- Jede Änderung einzeln gemessen
- Wirkung in Zahlen belegt
- Eine Änderung ohne Wirkung benannt

Wer drei Schalter gleichzeitig umlegt, hat nichts gelernt, nur etwas erreicht.

TAG 2 — WAS JETZT ANDERS IST

LATENZ

War eine Zahl → hat benennbare Ursachen



KERNE

Waren alle gleich → Ein Kern ist reserviert und nachgewiesen

PRIORITÄT

War ein Gefühl → ist eine Aussage über Deadlines

MORGEN: VON KONFIGURIEREN ZU SEHEN

HEUTE

Konfigurieren, dann
nachmessen, ob es
half

MORGEN

Sehen, wohin die Zeit
geht, bevor Sie etwas
ändern

PROGRAMM MORGEN

- Hardware-Zugriff und Interrupt-Handling in Echtzeit
- Debugging mit ftrace, perf und Trace-Werkzeugen
- Ihr Messsystem wird fertig